


Lecture 5

# CPU Organization and Operating System Structures

## Computer Organization and Digital Foundations

**Dr. Yatish Bathla**

**PhD.(Information Technology), Obuda University, Budapest**

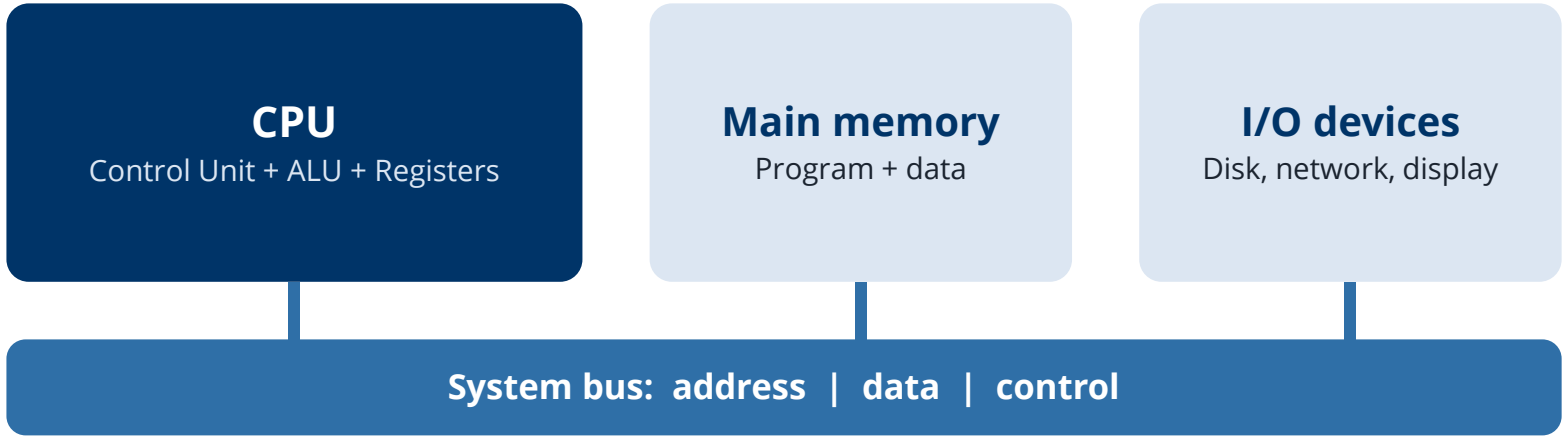
# Today's Roadmap

1. Describe the CPU components: control unit, ALU and registers
2. Trace an instruction through fetch, decode and execute
3. Explain kernel mode, user mode and system calls
4. Compare monolithic, layered, microkernel and hybrid OS structures

## 60-minute plan

| | |
|---|---|
| **0-3 min** | Orientation (slides 1-2) |
| **3-31 min** | CPU organisation (slides 3-11) |
| **31-48 min** | Operating system structures (12-17) |
| **48-60 min** | Practice, mistakes, summary (18-20) |

# The CPU in the Big Picture




**~3 GHz**
typical clock of a laptop CPU

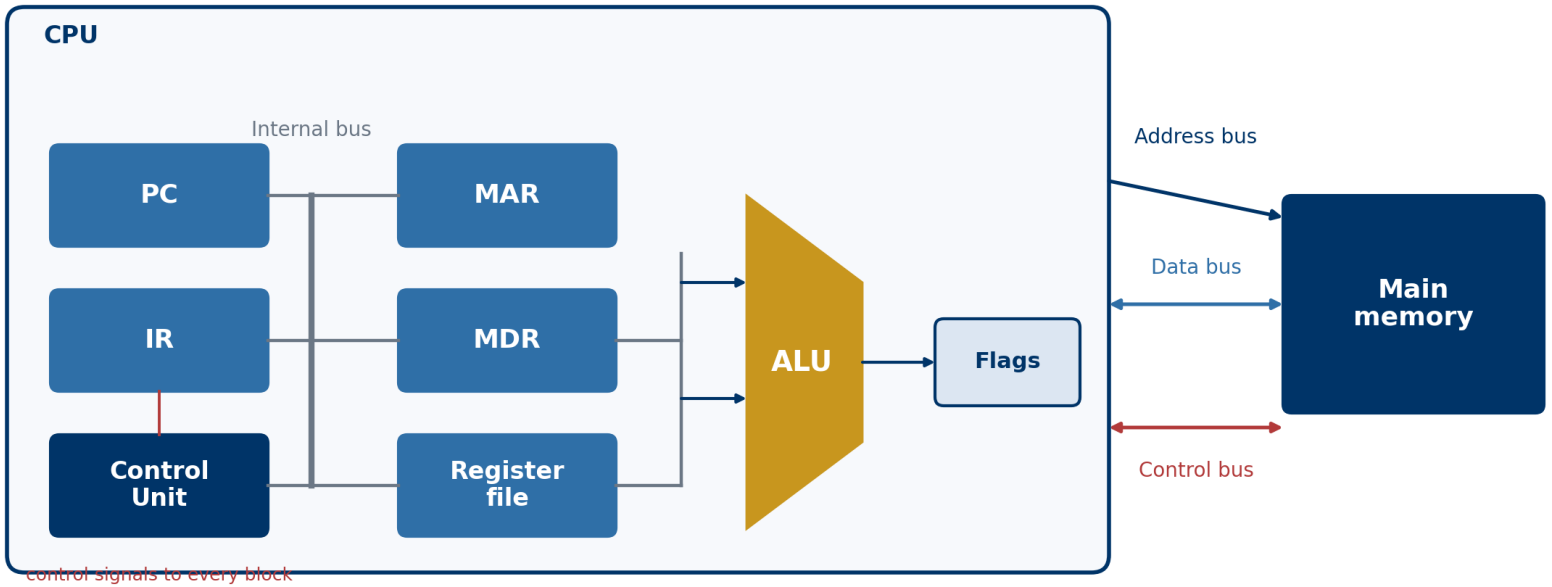
**4-16**
cores in a desktop CPU

**billions**
of instructions every second

- The CPU repeatedly **fetches** an instruction from memory, **decodes** it and **executes** it.
- Programs and data share main memory (stored-program concept).
- Modern CPUs add caches and several cores, but the basic organisation is unchanged.

**The CPU is the engine that executes the stored program, one instruction at a time (per core).**

# Inside the CPU: Datapath and Control



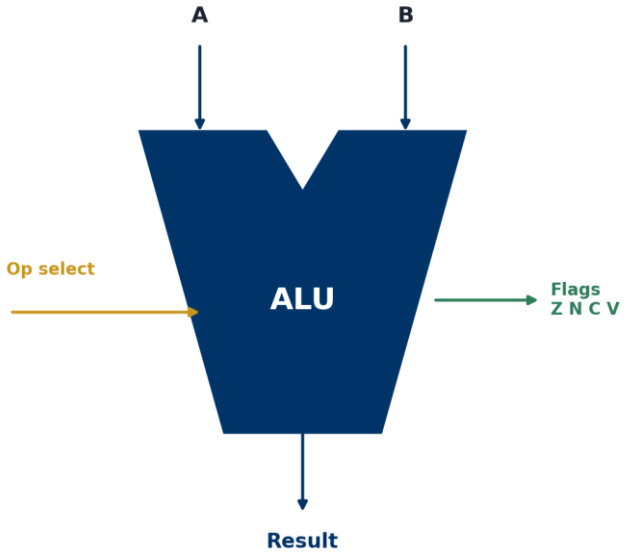

- **Datapath:** registers, ALU and internal buses that move and transform data.
- **Control unit:** reads the instruction and sends control signals.
- **Memory interface:** MAR and MDR talk to memory over the system bus.

# CPU Registers

| Register | Name | What it holds | Changes when |
|---|---|---|---|
| **PC** | Program counter | Address of the next instruction | After each fetch or branch |
| **IR** | Instruction register | The instruction being executed | At each fetch |
| **MAR** | Memory address register | Address of the memory location to access | Before each memory access |
| **MDR** | Memory data register | Data just read or about to be written | During memory access |
| **R0-Rn** | General-purpose registers | Operands, results, addresses | By instructions |
| **SP** | Stack pointer | Address of the top of the stack | On call / return, push / pop |
| **FLAGS** | Status register | Z, N, C, V result flags | After ALU operations |

**Registers are the fastest storage in the machine: access takes about one clock cycle or less.**

# The Arithmetic Logic Unit (ALU)




| Group | Operations |
|---|---|
| **Arithmetic** | ADD, SUB, INC, DEC |
| **Logic** | AND, OR, XOR, NOT |
| **Shift** | SHL, SHR |
| **Compare** | SUB, discard result |

**Example**

SUB R1, R2: if R1 = R2 the result is 0, so **Z = 1**. A following BEQ branch tests Z.

**Z** Zero
Result = 0

**N** Negative
Sign bit = 1

**C** Carry
Carry out of MSB

**V** Overflow
Signed result too big

# Instruction Set Architecture (ISA)

### A 32-bit instruction (RISC style)



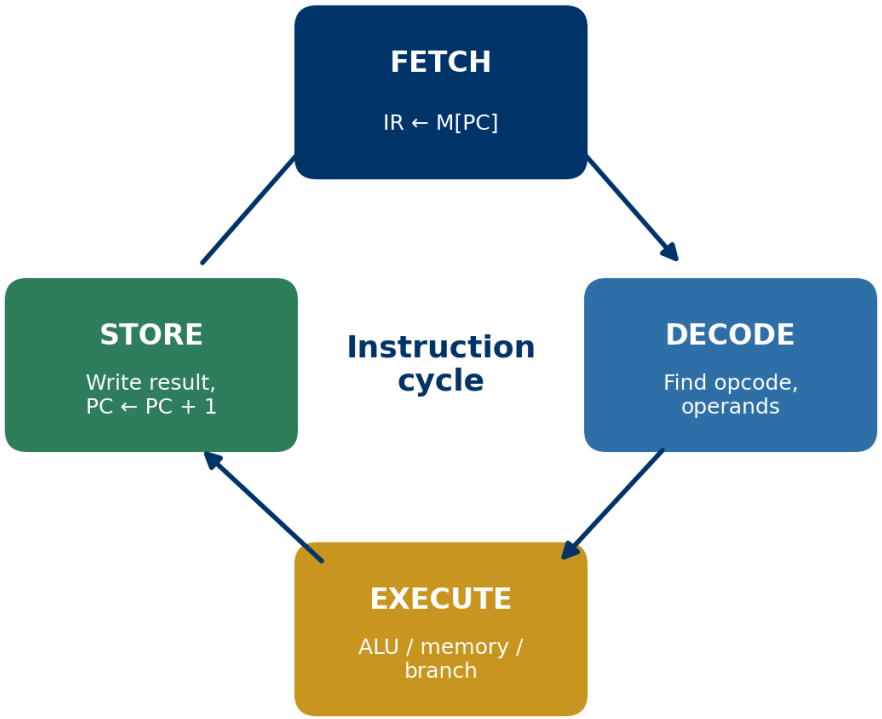

**ADD R1, R2, R3 means R1 ← R2 + R3**

- **ISA** = the programmer-visible interface: instructions, registers, addressing modes, data types.
- **Addressing modes:** register, immediate, direct, indirect, displacement.
- Examples: x86-64, ARM, RISC-V, MIPS.

| | RISC | CISC |
|---|---|---|
| **Instructions** | Few, simple | Many, complex |
| **Length** | Fixed | Variable |
| **Memory access** | Load/store only | Most instructions |
| **Examples** | ARM, RISC-V | x86 |

Same ISA, many designs: an Intel and an AMD chip run the same x86 programs.

# The Instruction Cycle




1. **Fetch**
   MAR ← PC; read memory into IR; PC ← PC + 1
2. **Decode**
   Control unit identifies the opcode and operands
3. **Execute**
   ALU computes, or memory is read/written, or PC changes
4. **Write back**
   Result stored in a register or memory; check for interrupts

# Worked Example: A Tiny Program

**Program in memory**

```
# address   instruction
100    LOAD  200    ; ACC = M[200] = 5
101    ADD   201    ; ACC = ACC + M[201] (7)
102    STORE 202    ; M[202] = ACC
```

| After | PC | ACC | M[202] |
|---|---|---|---|
| start | 100 | - | - |
| LOAD 200 | 101 | 5 | - |
| ADD 201 | 102 | 12 | - |
| STORE 202 | 103 | 12 | 12 |

| Step | Register transfer (fetch of LOAD) | Unit |
|---|---|---|
| T0 | MAR ← PC | Control |
| T1 | MDR ← M[MAR]; PC ← PC + 1 | Memory |
| T2 | IR ← MDR | Control |
| T3 | Decode: opcode = LOAD | Control |
| T4 | MAR ← IR.address (200) | Control |
| T5 | MDR ← M[MAR] (= 5) | Memory |
| T6 | ACC ← MDR | Datapath |

**Every instruction is a short sequence of register transfers controlled by the clock.**

# The Control Unit




| | Hardwired control | Microprogrammed control |
|---|---|---|
| **How it works** | Fixed logic gates and flip-flops | Control store (ROM) holds microinstructions |
| **Speed** | Fast | Slower (extra memory read) |
| **Flexibility** | Hard to change | Easy to change or fix by updating microcode |
| **Typical use** | RISC processors | Complex CISC instructions (e.g. x86) |

# Performance and Pipelining

**CPU time = IC × CPI × T**

IC = instruction count, CPI = cycles per instruction, T = clock period

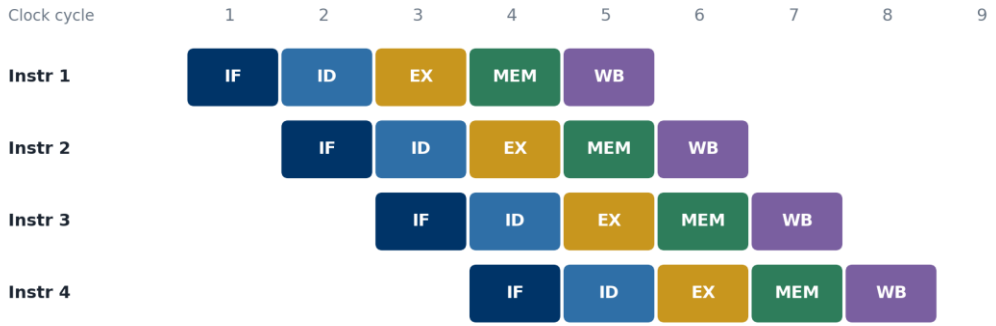
**Example**

IC = $2\times10^9$, CPI = 1.5, clock = 2 GHz

Cycles = $3\times10^9$

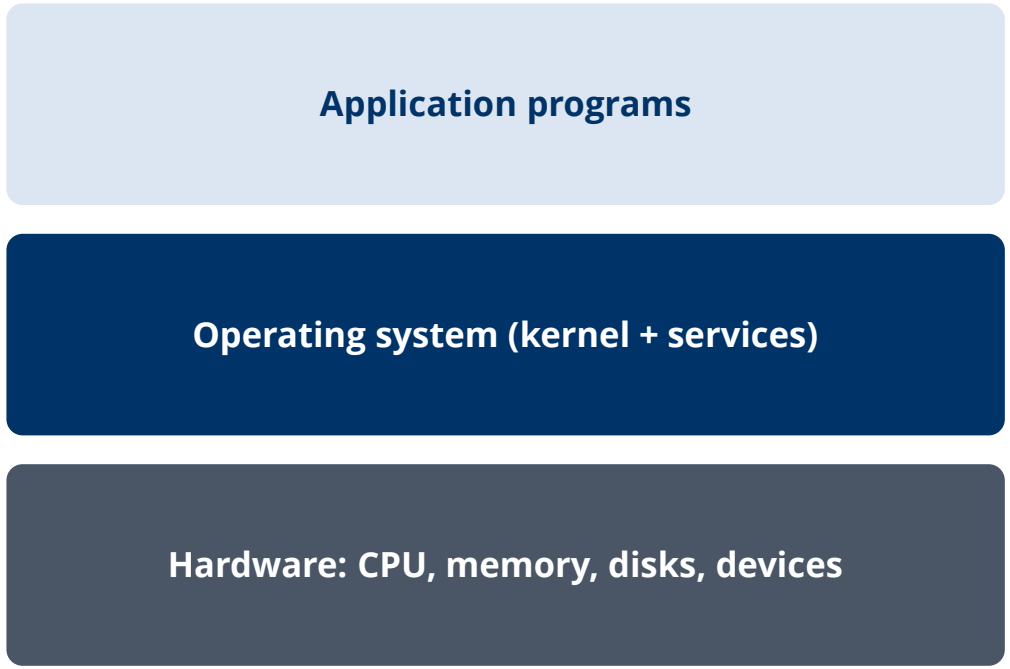
CPU time = $3\times10^9 / 2\times10^9$ = **1.5 s**

Lowering CPI (pipelining) cuts time without a faster clock.




- **Pipelining:** overlap fetch, decode, execute, memory, write-back of different instructions.
- 4 instructions: **8 cycles** pipelined vs 20 unpipelined.
- Hazards (data, control) can force stalls.

# What Is an Operating System?




Examples: Linux, Windows, macOS, Android

RM

### Resource manager

Shares CPU time, memory, storage and devices among programs.

AB

### Abstraction

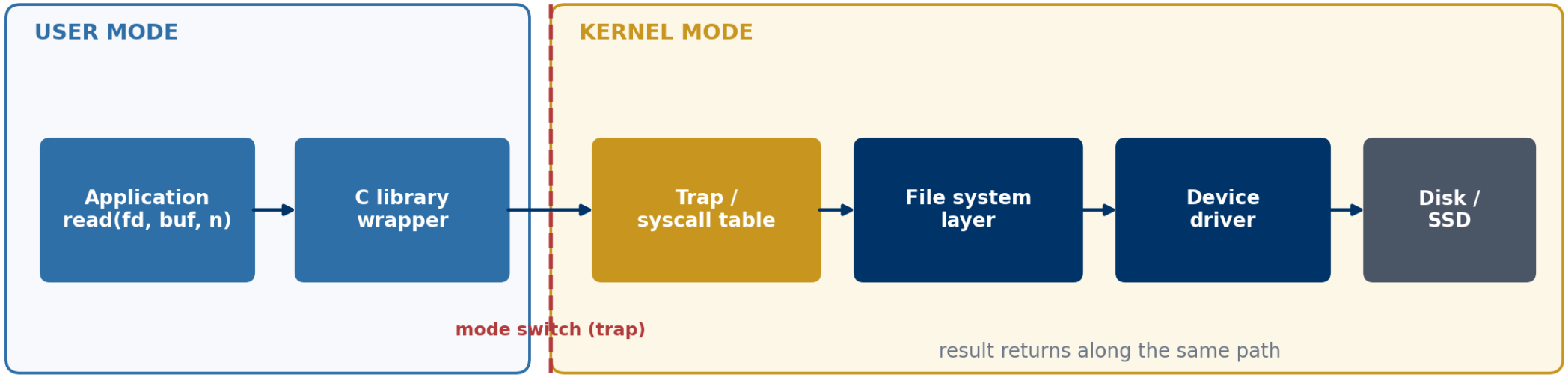
Provides files, processes and sockets instead of raw hardware.

PR

### Protection

Isolates programs from each other and from the kernel.

# Kernel Mode, User Mode, System Calls




**User mode:** restricted instructions only. **Kernel mode:** all instructions, including I/O and memory management. A **trap** switches mode and returns afterwards.

**Applications never touch hardware: they ask the kernel through system calls.**

# OS Services and System Calls

| Category | Linux examples | Purpose |
|---|---|---|
| **Process control** | fork, execve, exit, wait | Create, run, end programs |
| **File management** | open, read, write, close | Use files and directories |
| **Device management** | ioctl, read, write | Access devices |
| **Information** | getpid, time, uname | Query system data |
| **Communication** | pipe, socket, shmget | Talk to other processes |

**See a system call**

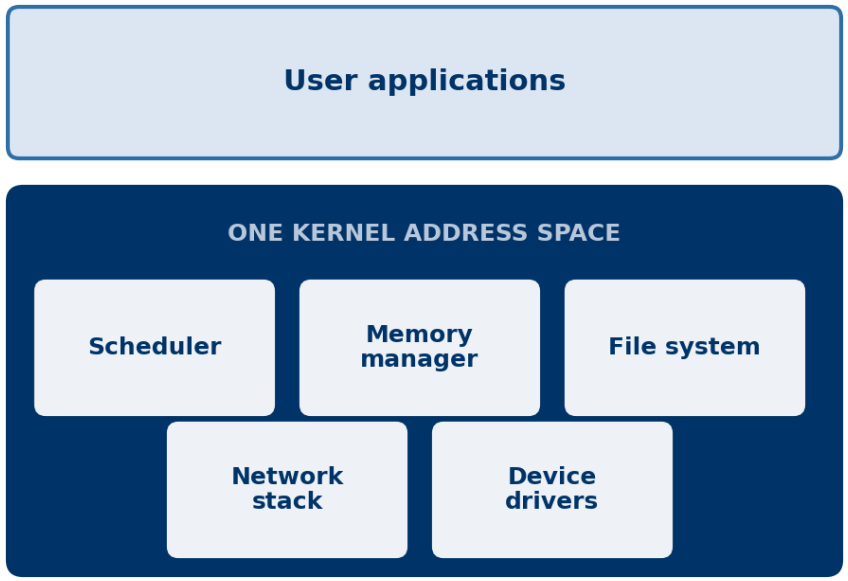
```
$ strace -e trace=write \
    echo hello
write(1, "hello\n", 6) = 6
```

**Library vs kernel**

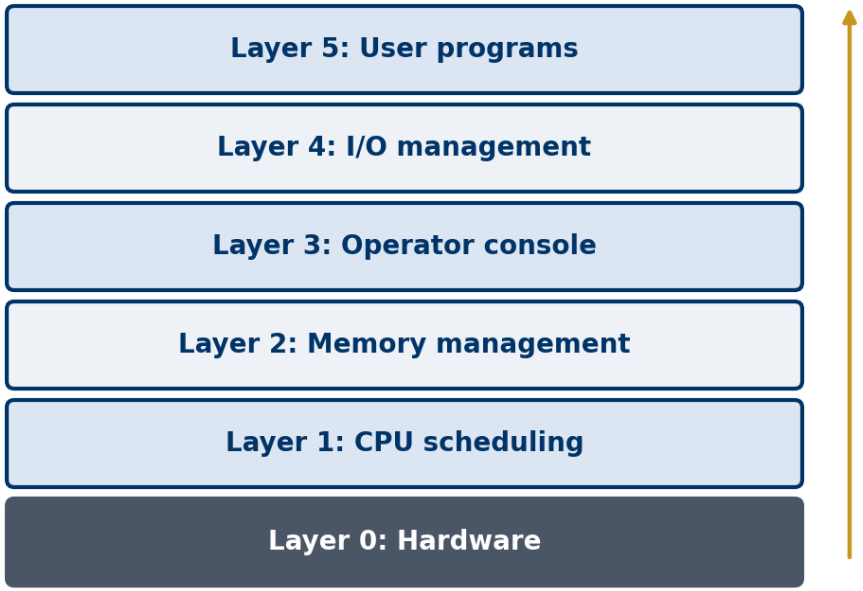
printf() → write() → kernel

**A small set of system calls is the entire interface between programs and the kernel.**

# OS Structures: Monolithic and Layered

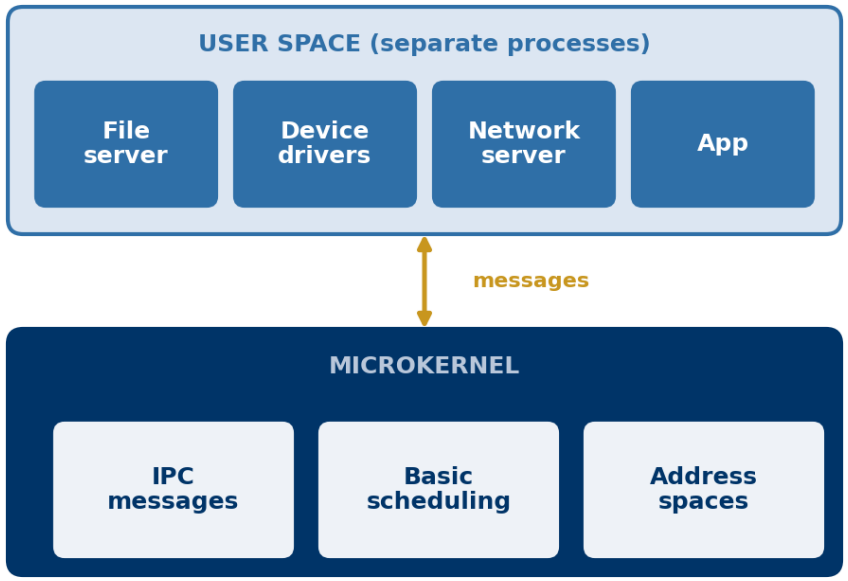



**Monolithic** (Linux, classic UNIX): fast, but a driver bug can crash everything.

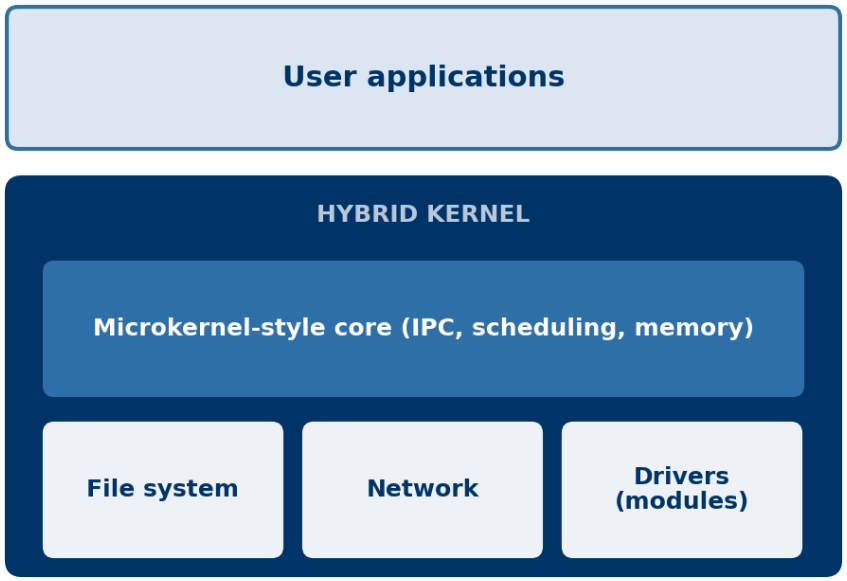



**Layered:** clear, easy to debug; each layer uses only the one below, adds overhead.

# OS Structures: Microkernel and Hybrid

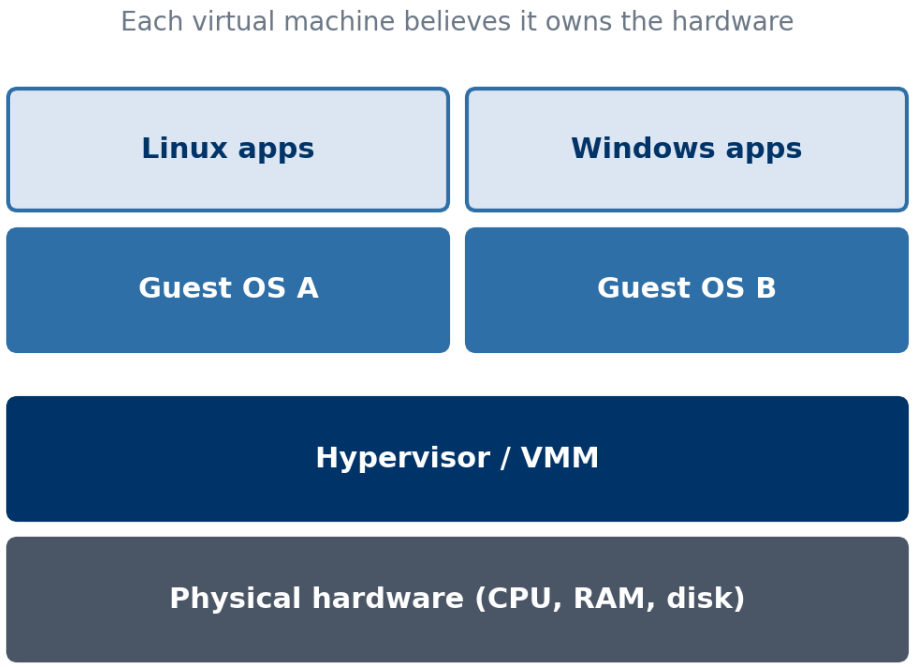



**Microkernel** (Minix, QNX, L4): tiny core, services in user space; robust, but message passing costs time.




**Hybrid** (Windows NT, macOS XNU): microkernel ideas with key services kept in the kernel for speed.

# Virtual Machines and Hypervisors




T1

## Type 1: bare metal

Hypervisor runs directly on hardware: Xen, KVM, VMware ESXi.

T2

## Type 2: hosted

Hypervisor runs on a normal OS: VirtualBox, VMware Workstation.

+

## Why use them

Isolation, server consolidation, cloud computing, testing.

# Practice Set: Apply What You Learned

| # | Question | Hint | Your answer |
|---|---|---|---|
| 1 | Which register holds the address of the next instruction? | Fetch | |
| 2 | Trace LOAD 200 (M[200] = 5), then ADD 201 (M[201] = 7). Value of ACC? | Slide 9 | |
| 3 | IC = $10^9$, CPI = 2, clock = 2 GHz. CPU time? | IC × CPI × T | |
| 4 | Why can a user program not execute I/O instructions directly? | Mode bit | |
| 5 | Which OS structure keeps device drivers in user space? | Small kernel | |
| 6 | What does a system call do that a function call does not? | Trap | |

**Challenge:** a 5-stage pipeline runs 100 instructions with no stalls. How many cycles in total, and how many without pipelining?

# Common Mistakes to Avoid

| | |
|---|---|
| ✗ PC holds the current instruction | ✓ PC holds the address of the next instruction; IR holds the current one |
| ✗ Higher GHz always means a faster CPU | ✓ Time = IC × CPI × T: consider all three factors |
| ✗ The kernel and the OS are identical | ✓ Kernel is the core; the OS also includes libraries and system programs |
| ✗ Microkernels are always slower, monolithic always better | ✓ Each is a trade-off between speed, safety and flexibility |

**Check the register, the formula and the mode: most errors come from mixing them up.**

# Summary and Next Steps

- The **CPU** = control unit + ALU + registers, connected to memory by buses.
- Each instruction runs **fetch → decode → execute → write back**.
- Performance = **IC × CPI × T**; pipelining lowers CPI.
- The **OS** manages resources, abstracts hardware and protects with **kernel/user mode**.
- Structures: **monolithic, layered, microkernel, hybrid**, plus virtual machines.

**Questions?**

**Lab**
Simulate circuits, study a register/ALU datapath, explore Linux OS structure.

**Reading**
H&P Ch. 1 (1.3, 1.9) and Sec. 2.4; Mano: Registers and Counters.

**Before the lab**
Install Logisim-evolution and have a Linux terminal ready.